

Paso 1: La "Hoja en Blanco" (Product Discovery y Requerimientos)

En una hoja de papel, el razonamiento estructurado parte de 4 preguntas fundamentales:

1. Problema real y usuario:

- *¿Qué dolor resuelvo?* Mi coach me exige registrar 2 postulaciones al día y mostrar evidencia clara con fechas y respuestas. Anotarlo a mano o en un Excel desordenado toma tiempo y no genera analíticas de valor.
- *¿Quién lo usa?* Yo (un desarrollador en búsqueda activa) en el celular de forma rápida.

2. Definición del MVP (Minimum Viable Product):

- *¿Qué es lo mínimo indispensable para que sirva hoy?*
 - Formulario rápido de carga (empresa, puesto, link, requisitos).
 - Registro de interacciones ("hoy mandé mensaje", "hoy me respondieron").
 - Cálculo de tiempos de respuesta.
 - Exportación a PDF para la coach.
- *¿Qué dejamos explícitamente para después?* Pagos, login multiusuario, matching avanzado con IA.

3. Entidades y Datos (Modelo Mental):

- Cada **Postulación** pertenece a una **Empresa** y tiene múltiples **Interacciones** a lo largo del tiempo.

Paso 2: Modelo de Datos Mental (Boceto Lógico)

Antes de elegir si usás PostgreSQL o MongoDB, se dibuja la relación de los datos:

Plaintext

[EMPRESA]

└─ nombre, web, rubro

[POSTULACIÓN]

└─ rol, fecha_aplicacion, estado (ENVIADA | CONTACTO | ENTREVISTA | RECHAZADA)

└─ requisitos_texto, skills_detectadas

└─ tiempo_respuesta_dias

└─ [HISTORIAL DE INTERACCIONES] (1 postulación tiene N eventos)

└─ 04/09: Postulación enviada

└─ 06/09: Mensaje enviado por LinkedIn al recruiter

└─ 10/09: Respuesta recibida -> (Dispara cálculo de días de demora)

Paso 3: Diagramas de Flujo Clave

Se grafican los dos caminos críticos del negocio: la postulación diaria y el cálculo cuando te responden.

Flujo A: Registro de una Postulación Diaria

Plaintext

[INICIO]

|



[Usuario ingresa datos en mobile (Empresa, Rol, URL, Requisitos)]

|



[Backend valida campos obligatorios]

|

└─ ¿Campos inválidos? —> [Retorna Error 400]

|

└─ ¿Válidos?

|



[Extraer keywords/skills de los requisitos]

|



[Guardar documento en MongoDB (Estado: "ENVIADA")]

|



[Retornar Confirmación 201 Created]

|



[FIN]

Flujo B: Registro de Interacción y Cálculo de Métrica

Plaintext

[INICIO]

|



[Usuario selecciona una postulación y toca "Me Respondieron"]

|

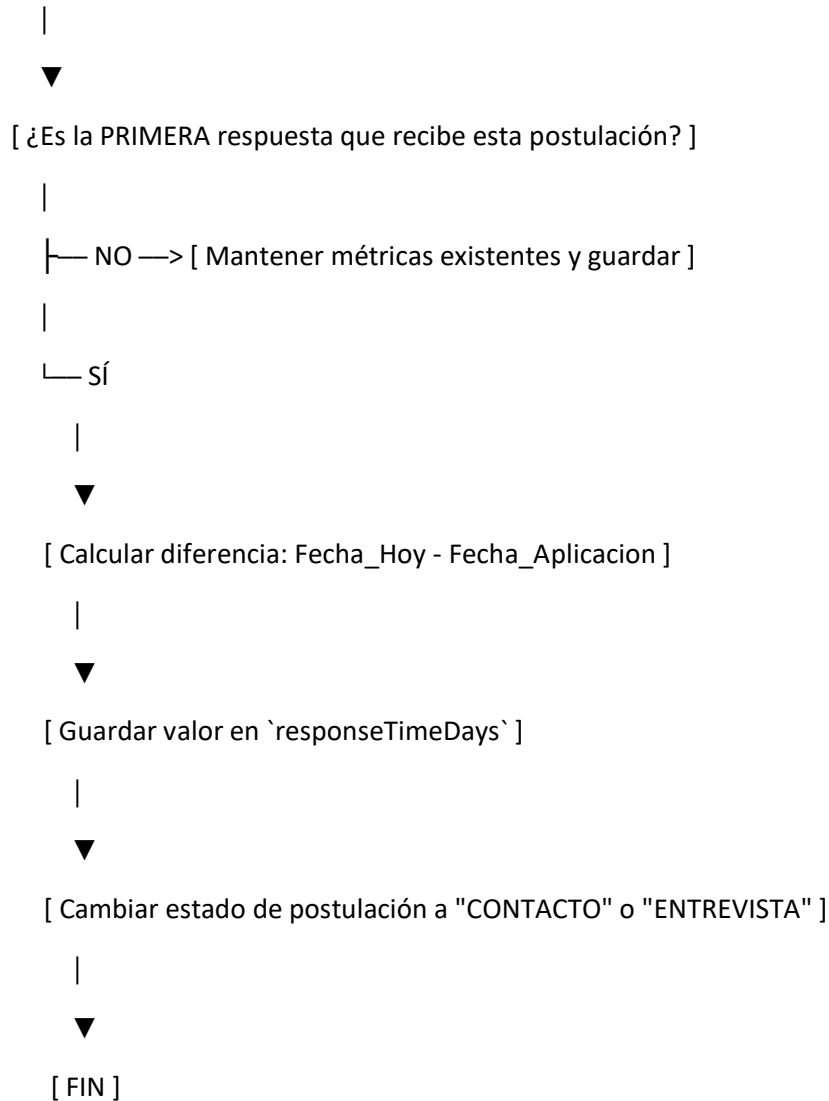


[Backend recibe evento (Tipo: "RESPUESTA_RECIBIDA", Fecha: Ahora)]

|



[Insertar evento en el array de interacciones]



Paso 4: Pseudocódigo de la Lógica de Negocio

El pseudocódigo te permite resolver los algoritmos sin preocuparte por la sintaxis de JavaScript o de Mongoose:

1. Pseudocódigo: Calcular tiempo de respuesta al registrar evento

Plaintext

ALGORITMO RegistrarInteraccion(postulacionId, tipoInteraccion, notas):

postulacion = BuscarEnBD(postulacionId)

SI NO postulacion ENTONCES:

RETORNAR Error("Postulación no encontrada")

nuevaInteraccion = {

tipo: tipoInteraccion,

fecha: OBTENER_FECHA_ACTUAL(),

notas: notas

}

AGREGAR nuevaInteraccion A postulacion.interacciones

// Lógica analítica

SI tipoInteraccion == "RESPUESTA_RECIBIDA" Y postulacion.tiempoRespuesta == NULO ENTONCES:

milisegundos = nuevaInteraccion.fecha - postulacion.fechaPostulacion

dias = REDONDEAR(milisegundos / (1000 * 60 * 60 * 24))

postulacion.tiempoRespuesta = dias

postulacion.estado = "CONTACTO"

GUARDAR(postulacion)

RETORNAR postulacion

FIN ALGORITMO

2. Pseudocódigo: Generación del Reporte PDF Semanal

Plaintext

ALGORITMO GenerarReportePDF(fechaInicio, fechaFin):

// 1. Filtrar postulaciones del período

postulaciones = OBTENER_POSTULACIONES(entre fechaInicio y fechaFin)

// 2. Calcular métricas resumidas

totalEnviadas = CONTAR(postulaciones)

conRespuesta = FILTRAR(postulaciones DONDE tiempoRespuesta != NULO)

tasaRespuesta = (CONTAR(conRespuesta) / totalEnviadas) * 100

promedioDiasRespuesta = PROMEDIO(conRespuesta.tiempoRespuesta)

// 3. Renderizar documento

datosReporte = {

 rango: { fechaInicio, fechaFin },

 metricas: { totalEnviadas, tasaRespuesta, promedioDiasRespuesta },

 listado: postulaciones

}

plantillaHTML = InyectarDatosEnPlantillaHTML(datosReporte)

bufferPDF = ConvertirHTMLaPDF(plantillaHTML)

RETORNAR bufferPDF

FIN ALGORITMO

Paso 5: Contrato de API (Endpoints planificados)

Antes de tocar Express, se listan las rutas HTTP necesarias:

- POST /api/applications (crear)
- GET /api/applications (listar/filtrar)
- POST /api/applications/:id/interactions (agregar mensaje/respuesta)
- GET /api/analytics/summary (métricas para la pantalla principal)
- GET /api/reports/pdf?from=...&to=... (descargar reporte)

Hacer este ejercicio previo te da un mapa mental absoluto: cuando te sentás a programar, no dudás qué esquema armar ni qué parámetros recibir, solo traducís este diseño a código.

¿Cómo va la terminal con el comando `pnpm run dev` y la ruta `/health`?